Indoor navigation among walking people: a measured pipeline in Isaac Sim

Dimitrios Gkiokas · dimitris.gkiokas@outlook.com · October 2026

Abstract. I built a simulation benchmark that takes one robot from reactive wandering to goal-directed autonomy in stages, and measures each stage against simulator ground truth. Worlds are generated from seeds: four rooms, doors of controlled width, furniture, and three people who stand still or walk scripted routes. A reactive navigator went from a terminal deadlock in all 25 worlds to none in 75 runs, kept its speed among walking people, and mapped all 25 worlds unattended (median 68.7% wall coverage at 99.0% occupied-cell precision). Nav2 missions on those maps reached 11 and 12 of 100 goals in the first complete tests, which left open whether the maps or the navigation stack set that ceiling. Perfect maps rendered from the world manifests answered it: under a stricter protocol the same configuration reached 6 of 100 goals on them and 9 on its own maps. A revised configuration, eight changes each traced to a failure in the logs, reached 79 of 100 goals on perfect maps and 44 of 100 on the robot's own maps in the same worlds, better in 24 and 18 of the 25 worlds, and 26 of 40 on ten worlds generated after it was frozen. The largest single change moves AMCL's base frame to the wheel axle: with the frame on the chassis centre, 88% of the localization jumps followed in-place turns. On self-built maps the remaining failures come from the areas the maps never covered. The ground-truth checks also exposed a start-up bug in Nav2 1.3.12 that reports goals reached while the robot stands at its spawn point. A follow-up that let the global costmap learn from the lidar did worse and was stopped after three worlds. All code, data and analysis are public.

1. Introduction

The question behind this project is practical: what does it take for a small wheeled robot to move with purpose through cluttered rooms while people walk around it? Rather than answer with one demonstration, I broke the problem into stages that each produce a number: how narrow a door the reactive layer can pass, what human traffic costs, how good a map the robot can build for itself under that traffic, and how far a standard navigation stack gets on that self-built map. The stages share one discipline. Every result is checked against the simulator's ground truth, never against the robot's own belief. That choice carries the main findings of the mission experiments: the stack's self-reported successes and its actual positions disagree by metres, perfect maps settled whether the maps or the stack limited the first results, and Nav2 itself reports arrivals at the spawn point after a failed transform. It also exposed a problem in my own scenario, described in Section 4.

The pipeline runs in NVIDIA Isaac Sim 6.0 on Windows with ROS 2 Jazzy (Nav2 1.3.12) in WSL2, on one RTX 2080 Ti, below the simulator's minimum specification. Real-time factors sat between 0.15 and 0.55. I mention the hardware because the engineering that keeps a pipeline split across two operating systems and two clocks consistent at half speed is part of the work: wall-clock stamps everywhere, file-based handshakes between the simulator and the ROS side, and batch machinery that ran 333 batch runs unattended.

2. The benchmark factory

A world is a function of a seed. The generator splits a 20 × 20 m arena into four rooms with two crossing partition walls, cuts one door per half-wall with widths drawn from two bands (two anchor doors at 0.72–0.95 m, the other two at 0.50–0.78 m), scatters furniture, and writes a manifest CSV: every wall, door and obstacle with coordinates, plus walking routes for the three people. The manifest is the ground truth that every later measurement joins against. People walk their routes at constant speed with per-waypoint pauses and do not react to the robot. In the last walking condition one person per world is routed through a door to a neighbouring room (the widest one with a clear route), which turns that doorway into a contested resource. The camera sees the people's character meshes, the lidar a 0.25 m cylinder that moves with each person.

The robot is a 0.42 m differential-drive body. Its lidar is a 360-ray PhysX raycast published to /scan at about 20 Hz. Odometry integrates ground-truth motion with multiplicative noise (3% linear, 5% angular), which produced measured drifts up to 1.57 m in the mapping runs. A logger records robot and people ground-truth positions at 10 Hz into one CSV per run. Batches run unattended: the simulator side regenerates worlds and signals the WSL side through files. A deadlock watchdog ends a run when its whole position trail stays inside a 0.5 m box for 180

wall-clock seconds. An earlier watchdog compared only the trail's endpoints and killed 16 of 25 healthy runs whose paths looped back near old positions. The post-mortem of that batch is what fixed the rule.

3. Reactive navigation vs door width

The reactive layer is a follow-the-gap navigator over the raw scan with a stack of recovery behaviours, each added after a failure observed in an earlier version, plus a camera-based person brake (YOLOv8n). Its predecessor, the same core without the recovery layers, reached a terminal deadlock in all 25 worlds (median onset 24 s). Version 2.9.1 completed 75 runs of 20 wall-clock minutes, static and with walking people, without a terminal deadlock. At doors, success depends on width. Of the doors it approached, the robot crossed 12 of 14 between 0.80 and 0.95 m, 10 of 17 between 0.66 and 0.80 m, and 2 of 7 below 0.66 m, the nominal minimum (0.42 m body plus 0.12 m inflation on each side). Only 38 of the 100 doors were approached, so Figure 1 shows a trend rather than a sharp threshold. The generator samples the harder doors around that boundary on purpose, so most worlds have a door near it.

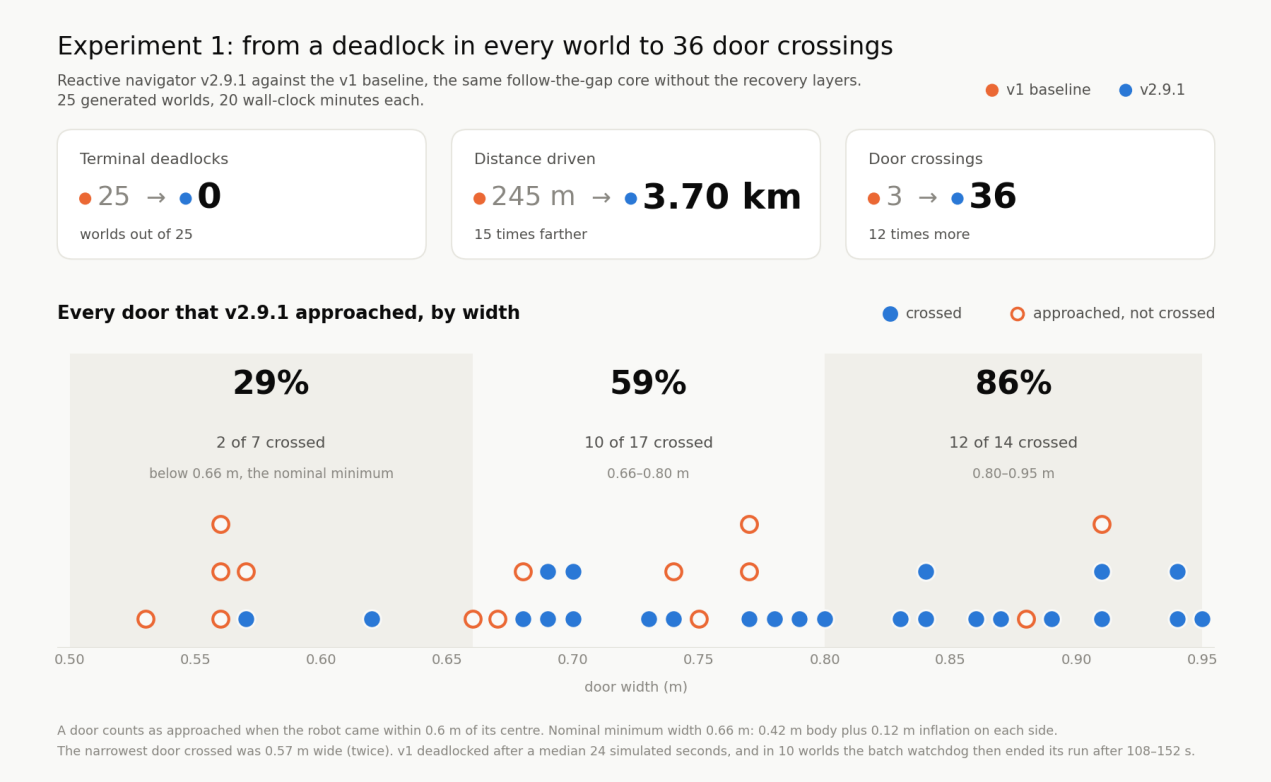


Figure 1. Top: the v1 baseline against v2.9.1 in the same 25 worlds. Bottom: every door v2.9.1 approached, at its width, filled when crossed, with the share crossed in three width groups.

4. What walking people cost

The same 25 worlds ran under three conditions: the three people standing still, 1–3 of them walking routes inside their rooms, and the same with one person per world crossing rooms through a contested door. That is 75 runs of 20 wall-clock minutes, with identical geometry per seed because the generator draws all routes after placement. Median exploration speed did not move: 13.8, 13.8 and 13.9 m/min. No run ended in a terminal deadlock. Door crossings rose from 7.9 to about 10 per simulated hour, and encounters in the zone of the contested door rose from 26 to 54, while the robot kept using that door (17 crossings with and without the cross-room walker).

The ground-truth logs also show something I first missed. The walkers follow their routes without yielding, and they walk into the robot. The robot is about 0.4 m wide and each person's collision cylinder has a 0.25 m radius, so their shapes meet at roughly 0.40–0.45 m centre to centre, and most walking runs had a closest pass below 0.45 m. A hard push appears in the log as the robot moving faster than 1 m/s, against a top speed of 0.4 m/s, with a person within 0.5 m. In 39 of the 41 such events across all walking batches the robot was displaced away from the person. Hard pushes happened in 7 of the 25 same-room runs and 12 of the 25 cross-room runs, mostly from the side or

from behind, and never with the people standing still. These counts are a lower bound, since lighter contacts do not move the robot that fast. The closest pass of the 75 runs, 0.27 m centre to centre, was one of them: the cross-room walker came up behind the robot at a 0.67 m door and pushed it 3 m. The person brake looks only at the middle third of the forward camera image, so it could not react. My earlier near-contact threshold of 0.35 m sits below the distance at which the two shapes meet, which is why it flagged only one of these events. The contacts say as much about the scripted walkers as about the navigator, but a navigator whose person brake only looks forward has no way to avoid them.

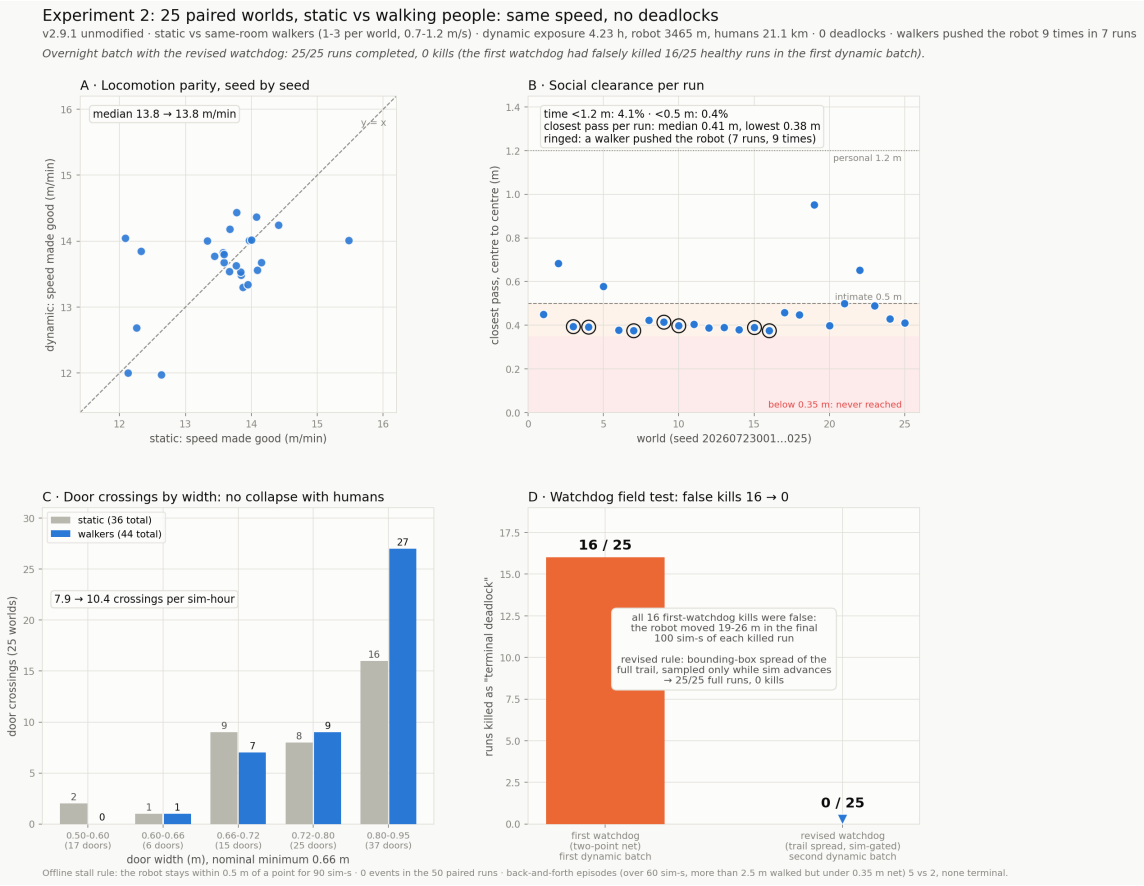


Figure 2. Static vs walking people across 25 worlds: speed seed by seed, closest pass per run (ringed where a walker pushed the robot), door crossings by width, and the watchdog field test.

5. Mapping while people walk

Stage three hands the same wandering robot to slam_toolbox: 25 worlds, 20 wall-clock minutes each (about 10 simulated minutes), odometry noise and walkers active, one map saved per world by an orchestrator that restarts SLAM between worlds. All 25 maps were saved without intervention in one unattended session. Two results matter later. First, I found no sign that walking people degrade the maps: time spent within 1.2 m of a person is barely correlated with coverage ($r = -0.10$), and in a single-world test the walking people left 0.16 m^2 of phantom obstacles in the map, against 0.36 m^2 with the same people frozen in place. Second, the wander policy is the bottleneck: median wall-surface coverage is 68.7% (range 51.5–94.5%), against 99.0% median occupied-cell precision. The robot maps what it visits accurately. It just does not visit enough. Doors come out narrower on the map than in truth by a median of 5 cm, which matters at a planner's inflation radius. Measured odometry drift is barely related to map precision ($r = -0.17$, $n = 25$).

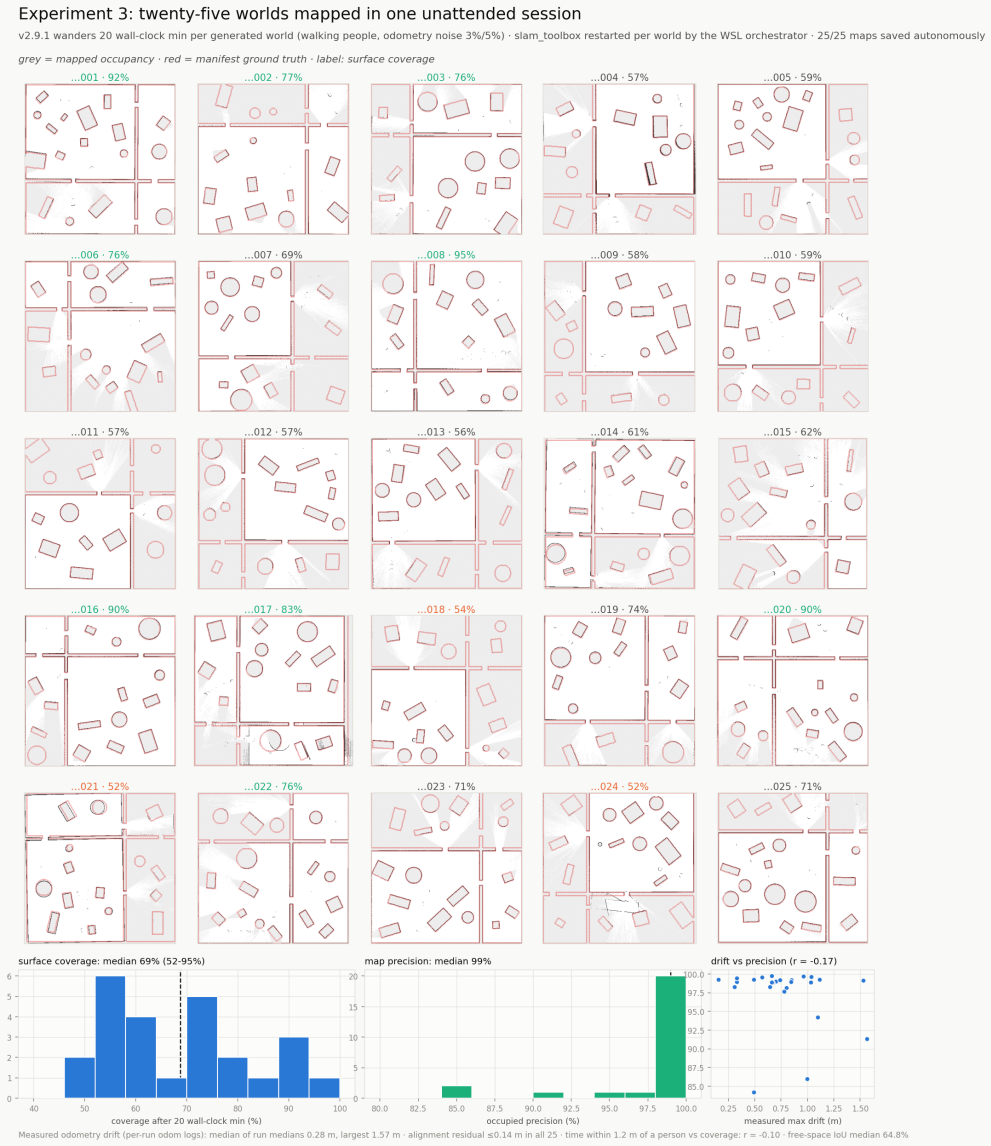


Figure 3. Twenty-five worlds mapped in one unattended session. Grey: mapped occupancy. Red: manifest ground truth. Per-world label: wall-surface coverage.

6. First missions on self-built maps

The next stage gives Nav2 (AMCL, NavFn, DWB) each world's self-built map and a four-goal tour: the centres of the three other rooms, then home. Before running anything I computed, per world, which goals are reachable at the planner's 0.30 m radius on the map and on the ground-truth geometry (a breadth-first search over the inflated grid). The check says 85 of 100 goals are feasible on the maps and 82 in truth, and flags four worlds as cages whose spawn room has no door the conservative planner will accept.

With baseline AMCL settings (v8) and 240 wall-clock seconds per attempt, the stack truly reached 11 of 100 goals, where truly means the ground-truth position at the moment of reported success lies within 1.5 m of the goal: 7 of the 60 cross-room goals the check calls feasible, plus 4 returns home, three of them in worlds the robot never left. Six more goals were reported reached and were not, in four worlds. Three of these were reported while the robot still stood at its spawn point, 0.1 to 5.3 s after the goal was sent (Section 11). The four predicted cage worlds refused every cross-room goal, as they should. I then retuned AMCL for sparse maps (v9: 500–4000 particles instead of 300–1500, a wider motion model, more tolerance for beams that the map cannot explain) and re-ran all 25 tours. The true count moved from 11 to 12. The three arrivals reported at a wrong position disappeared and the failures changed form: the robot timed out or stayed wedged (wedged worlds 4 to 8). One v9 tour (world 007) reported all four goals within 4.3 s without moving, the start-up bug again, and is excluded.

These first tests left a hypothesis: that the maps set this ceiling, since 20 minutes of wandering leave parts of each world unmapped, although coverage correlated only weakly with real arrivals (r = 0.28 and 0.39). Section 7 tests it.

7. Map or stack?

To separate the two, I rendered a perfect map of every world from its manifest, at the resolution and in the frame of the SLAM maps, and ran the first configuration (v9) and a revised one (Section 8) on both kinds of map in the same 25 worlds. The protocol changed for all four arms. Each attempt gets 120 simulated seconds, read from the simulator's own log. A failed goal is retried once after 10 simulated seconds. A real arrival is a reported success within 0.6 m of the true position. Every mission log row records the configuration, the kind of map and the md5 of the parameter file. The world is the unit of analysis, because the four goals of a tour are not independent, so intervals come from a bootstrap over worlds and comparisons are paired by world.

v9 reached 6 of 100 goals on perfect maps (95% interval 2–11) and 9 on SLAM maps (4–15). The revised configuration reached 79 (69–88) and 44 (34–55). Compared world by world (Figure 4), on perfect maps it was better in 24 worlds and worse in none (+2.92 goals per world, sign test $p = 1.2 \times 10^{-7}$), on SLAM maps better in 18 and worse in 1 (+1.40, $p = 7.6 \times 10^{-5}$). For v9 the map made no difference (–0.12 goals per world, p = 0.51), because the stack failed before the map could matter. For the revised configuration a perfect map was worth 1.40 goals per world (better in 20 worlds, worse in 3, $p = 4.9 \times 10^{-4}$). The ceiling of the first tests was mostly the navigation stack.

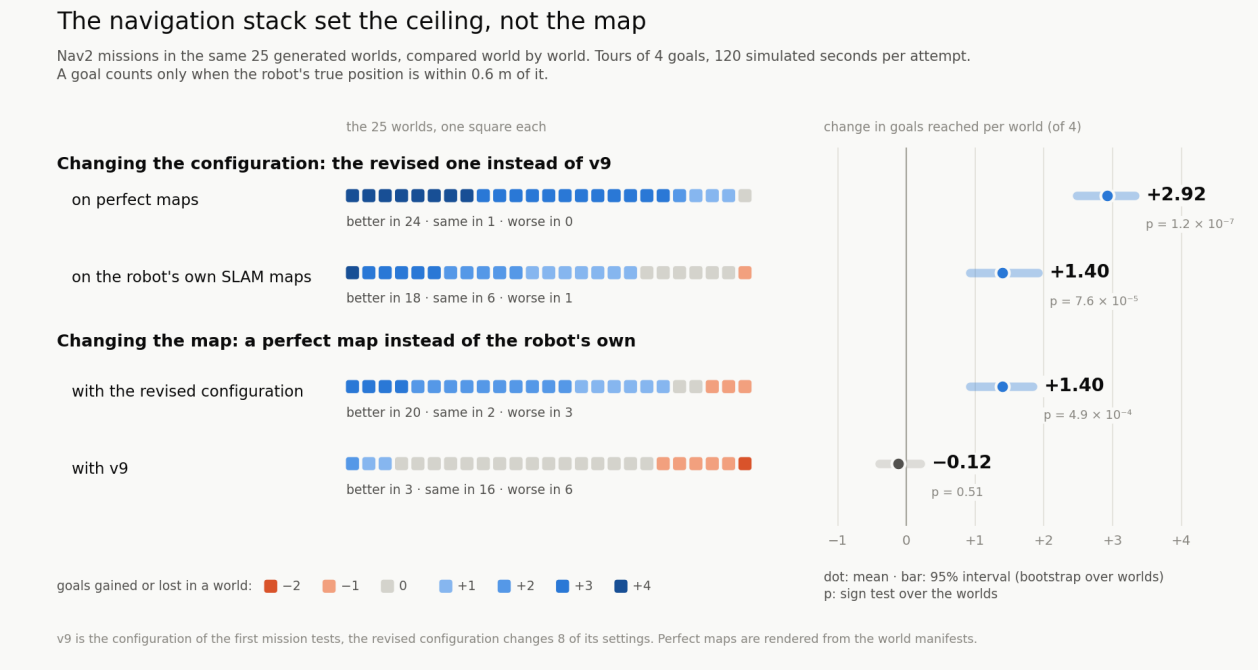


Figure 4. The 2 × 2 on 25 worlds, one row per paired comparison. Squares: the 25 worlds, shaded by the goals gained or lost in each. Right: mean change per world with its 95% bootstrap interval and the sign test.

8. Eight changes

The revised configuration changes eight settings of v9, each answering a failure found in the experiment 6 logs and the ground truth, and was frozen after pilots on six separate development worlds. (1) The controller, local costmap and recoveries work in a frame on the wheel axle, the point the differential drive turns about, 0.10 m behind the lidar. (2) The footprint is the true body outline plus 2 cm instead of a 0.22 m circle that missed the chassis corners. (3) The planner radius drops from 0.30 to 0.25 m. v9 often stopped closer to an obstacle than its planner accepts as a start, after which no later goal could be planned. (4) The velocity smoother lets reverse commands through. (5) The collision monitor leaves the command path. Its 0.23 m stop circle froze the robot for the rest of a tour after one close pass. (6) The behaviour tree replans only when the path becomes invalid, instead of every second. (7) AMCL tracks the wheel axle (7a) and injects no random particles (7b). (8) DWB no longer issues commands inside the simulated drive's deadband, where 39% of v9's commands fell.

Change 7a is the largest. With AMCL tracking the chassis centre, every in-place turn moves the tracked frame sideways on a 0.10 m circle, and Nav2's differential motion model reads each such step as a translation at a 90° bearing, spreading the particles by about 45° and 1 m per update. Of the 299 AMCL updates in experiment 6 whose error grew by more than 0.5 m, 263 (88%) came right after an in-place turn (Figure 5). With the revised configuration there were 12 such jumps in 7550 updates.

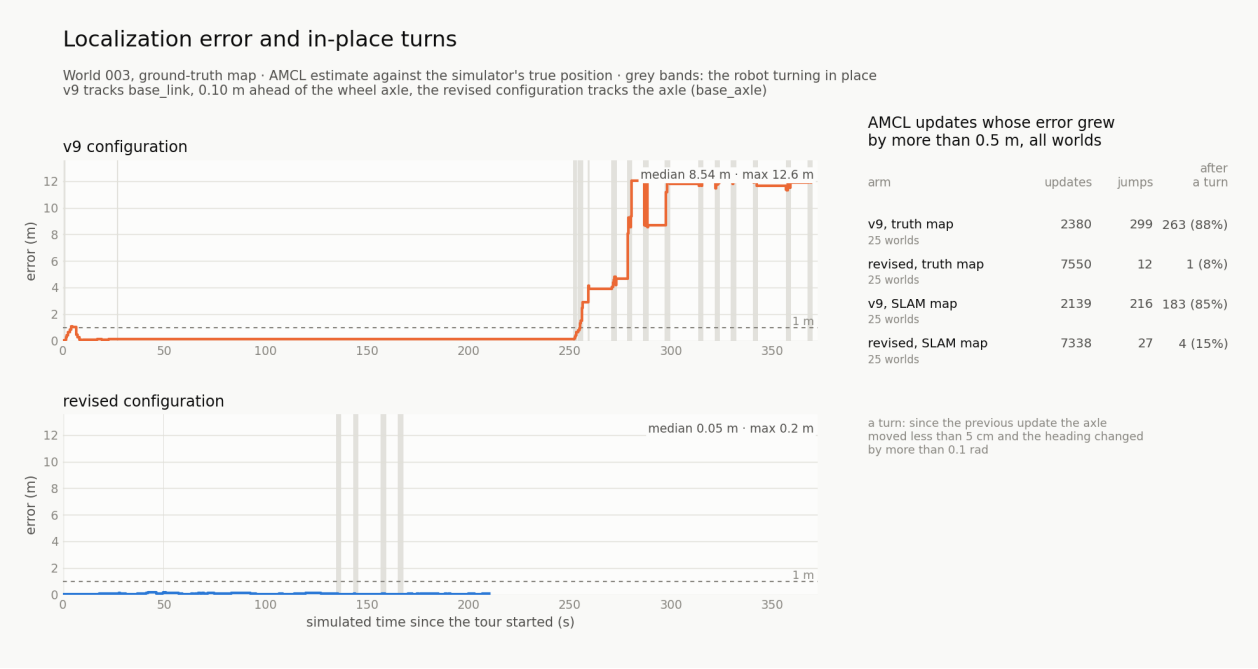


Figure 5. AMCL error over one tour of world 003, v9 and revised, perfect map. Grey bands: in-place turns. Right: error jumps over all worlds and how many followed a turn.

9. Checks

Because the changes were designed from logs of the benchmark worlds, I ran three checks on perfect maps. On 10 worlds generated after the configuration was frozen it reached 26 of 40 goals (65%, 95% interval 45–82%), the better estimate for unseen worlds. Two of the 14 misses are the start-up bug of Section 11. Eight benchmark worlds run again gave 27 of 32 goals against 20 of 32 in experiment 8. Three of them had been picked for doing badly, so part of the change is regression to the mean, and the rest shows how much one tour varies between runs. Finally, the same 18 worlds with change 7a undone lost 0.94 goals per world (95% interval 0.39–1.50, worse in 11 worlds, better in 2, sign test $p = 0.022$), and the median AMCL error of a tour exceeded 1 m in 3 of the 10 new worlds.

10. What still fails on self-built maps

Every failed attempt gets one primary label from the logged evidence (Figure 6). v9 failed mostly by losing localization (113 of 188 failed attempts on perfect maps). The revised configuration on perfect maps most often ran out of recoveries or time after driving (26 of 55). On SLAM maps two mechanisms account for most of its 120 failed attempts. Localization drifted where the map is blank (48): AMCL updates with more than 1 m of error happened where a median 39% of the map within 3 m had been seen during mapping, against 83% for the other updates. And the planner routed through unexplored space (38): Nav2 reads unseen cells as free, and the shortest route on the map crossed unseen cells that are blocked in truth in 64% of the failed attempts, against 24% of the successful ones. The revised configuration touched walls and furniture 8.8 and 13.2 times per kilometre on perfect and SLAM maps, against 88 for v9. Walker pushes rose in number (15 against 7 on perfect maps) as the robot drove six times farther among people who do not yield.

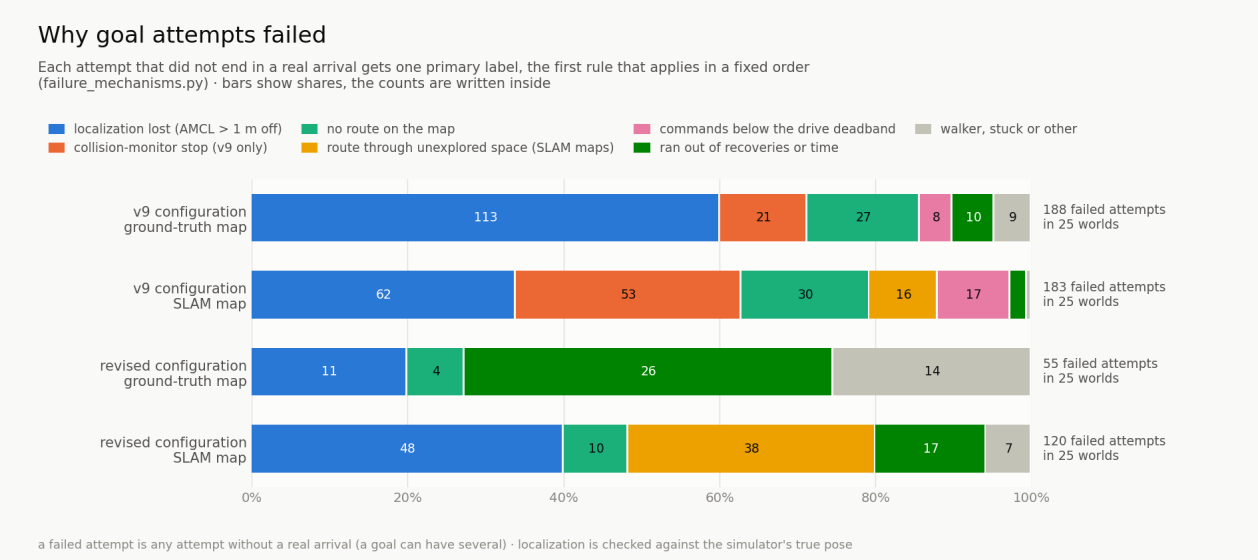


Figure 6. The primary failure mechanism of every failed goal attempt, four arms.

A post hoc follow-up (experiment 9) targeted the second mechanism: unseen cells stay unknown and costly for the planner, and the lidar marks the global costmap, so that a wall seen during the tour invalidates a route through it. I froze it before running and stopped the batch by choice after three complete worlds, because it was clearly worse: 4 of 12 goals against 9 of 12 for the revised configuration in the same worlds. In world 001 the planner failed for more than 30 seconds from a position with a free route on both maps, most likely because live marks such as walkers or door edges closed the passage.

11. A start-up bug in Nav2

Five tours in the data report arrivals the robot never drove, all at the start of a tour with the robot still at its spawn point. The Nav2 logs kept for two of them show the controller server failing to transform the goal, because the map to odom transform is older than its tolerance, and reporting the goal reached in the same millisecond. The behaviour matches a goal check that continues with an empty pose at the origin of the odometry frame, where the robot starts. The current Jazzy source aborts with a TF error in this case, and the installed 1.3.12 library lacks that error message. The ground-truth judge counts all such reports as false arrivals.

12. Main conclusion

The project has three results. The first is the navigator. The same follow-the-gap core, with recovery layers added one observed failure at a time, went from a terminal deadlock in all 25 worlds (v1, median onset 24 s) to none (v2.9.1), travelled 3.7 km instead of 245 m in the same worlds, kept its behaviour among walking people and explored all 25 worlds of the mapping batch unattended. Its limits are measured as well: a person brake that only looks forward, and a median wall coverage of 68.7% in 20 minutes. The second is the revised Nav2 configuration, which raised real arrivals from 6 to 79 of 100 goals on perfect maps and from 9 to 44 on self-built maps, with 65% on unseen worlds. The third is the way all of this was measured. Checking every result against ground truth changed the conclusion at each stage: it caught arrivals the stack reported but never made, a start-up bug in Nav2, a watchdog that killed 16 healthy runs, a mapping run with no odometry noise and flaws in my own walker scenario, and it answered the question the first tests left open: the ceiling was the stack, not the maps.

13. Limits and future work

Twenty-five worlds per condition, plus ten new worlds, support the differences reported here, not fine statistics. The feasibility check approximates the planner. The first missions used a 1.5 m arrival threshold and wall-clock budgets, and the real-time factor of the v9 batch fell to 0.15–0.27 in part of the worlds, so some of its attempts had as little as 36 simulated seconds. The later missions use 0.6 m and simulated time, and the two periods are never compared directly. The walkers do not react to the robot, and pushes are detected from the robot's motion, so lighter contacts are missed. Everything ran in one simulator on one machine, with wall-clock timestamps that do not emulate a real robot's clock discipline. The next step on self-built maps is coverage: frontier-based exploration in place of reactive wander would fill the blank areas where localization drifts and the planner finds routes that do not exist. On the social side, walkers that react to the robot and a navigator that watches its rear with the 360-degree lidar would address the contacts of Section 4.